

Document number: P0829R0
Date: 2017-10-14
Reply-to: Ben Craig <ben dot craig at gmail dot com>
Audience: Library Evolution Working Group

I. Introduction

C++ defines a "freestanding" implementation that is intended to be used on systems with limited operating system support [intro.compliance]. The standard has a small set of headers and declarations that are required to be present in a freestanding implementation.

Some language features don't work well, or at least work differently, in system programming. Exceptions, RTTI, heap management, global state, thread-local storage, and floating point operations are all problematic.

This proposal seeks to refine the list of headers and declarations to include all parts of the hosted implementation that don't use problematic features. This includes additions, modifications, and removals from the current set of freestanding headers and declarations.

This is intended as an early paper to seek general direction and opinions.

II. Motivation

Systems programmers want to sort things. They want to use move semantics. They may even want to bundle the arguments of a variadic template function into a tuple. These are all reasonable things to do on a system with no operating system and kilobytes of storage. The C++ standard even has reasonable specifications for these operations in the context of a tiny, OS-less system. However, systems programmers must currently rely on either hand-rolled code or implementer extensions in order to get these facilities.

Systems programmers don't have a guide as to what C++ library facilities will work without trying them. The standard says `atomic_load` will work; `memcpy` will probably work; but will `stable_sort`? Standardizing the subset of implementable C++ that is usable in a freestanding environment would provide clarity here, and help to educate systems programmers.

There were some presentations at recent CppCon where others made a more full featured C++ work in the Linux and Windows kernels [Quinn2016] [Baker2017]. In both of these cases, C++ was being used in a sandboxed / pseudo-virtualized way. C++ code (with exceptions and RTTI) was being run in the kernel context, but very few calls were made between the C++ code and the operating system kernel. The C++ code was in a guest operating system. This proposal should make it reasonable to have C++ code interact closely with the internals of a host operating system, perhaps in the context of a driver.

The current state of freestanding implementations is rather sad. Of the "big three" standard libraries (`libstdc++`, `libc++`, and Microsoft's Visual Studio STL), only `libstdc++` has any relevant mention of "freestanding" or "hosted". In practice, users take a hosted implementation of C++ and use it as-is in situations where it was never intended. This means that all the headers tend to be available, but only a few of the headers actually work. The parts that "should" work in a freestanding system, as specified by the C++17 standard, don't always work, while parts that aren't specified work fine. For example, just creating an instance of `std::bad_alloc` (without even throwing it!) causes linker errors in the Windows kernel with Visual Studio 2017, and it causes linker errors on a bare metal ARMv5 target with gcc 4.8.

```
Visual Studio 2017, x64 Kernel
cppCode.obj : error LNK2019: unresolved external symbol "void __cdecl operator delete(void *,unsigned __int64)" (??3@YAXPEAX_K@Z) referenced i
cppCode.obj : error LNK2019: unresolved external symbol __std_exception_destroy referenced in function "public: virtual __cdecl std::exceptio
```

```
gcc-4.8, ARMv5 bare metal
libgcc.a(unwind-arm.o): In function `get_eit_entry':
  unwind-arm.c:(.text+0x20c): undefined reference to `__exidx_end'
  unwind-arm.c:(.text+0x210): undefined reference to `__exidx_start'
libc.a(lib_a-abort.o): In function `abort':
  abort.c:(.text+0x10): undefined reference to `_exit'
libc.a(lib_a-sbrkr.o): In function `_sbrk_r':
  sbrkr.c:(.text+0x18): undefined reference to `_sbrk'
libc.a(lib_a-signalr.o): In function `_kill_r':
  signalr.c:(.text+0x1c): undefined reference to `_kill'
libc.a(lib_a-signalr.o): In function `_getpid_r':
  signalr.c:(.text+0x44): undefined reference to `_getpid'
libc.a(lib_a-writer.o): In function `_write_r':
  writer.c:(.text+0x20): undefined reference to `_write'
libc.a(lib_a-closer.o): In function `_close_r':
  closer.c:(.text+0x18): undefined reference to `_close'
libc.a(lib_a-fstatr.o): In function `_fstat_r':
  fstatr.c:(.text+0x1c): undefined reference to `_fstat'
libc.a(lib_a-isatty.o): In function `_isatty_r':
  isatty.c:(.text+0x18): undefined reference to `_isatty'
libc.a(lib_a-lseekr.o): In function `_lseek_r':
  lseekr.c:(.text+0x20): undefined reference to `_lseek'
libc.a(lib_a-readr.o): In function `_read_r':
  readr.c:(.text+0x20): undefined reference to `_read'
```

`std::sort` works just fine on both targets though.

III. Scope

The current scope of this proposal is limited to the freestanding standard library available to systems and embedded programming.

This paper is currently concerned with the divisions of headers and library functions as they were in C++17. I have not seen the paper that proposes which parts of the standard library go into which [sub-]modules, much less a paper that says which [sub-]modules will be freestanding. A later paper may attempt to do that.

I could see the scope increasing to the standard library availability on GPUs. It also occurs to me that the list of standard library functions suitable for a freestanding implementation is likely a large subset of the list of functions that could be marked `constexpr`, and conditionally `noexcept`. This paper will not attempt to address those items though.

IV. Impact on the standard

Rather than list all of the facilities available to a freestanding implementation in one place, as is currently done in [compliance], the standard would tag each header, class, or function that is available in a freestanding implementation. I expect this to be a large number of small edits, but the edits. I expect the edits would have easy to understand ramifications throughout the standard.

There is precedent for this kind of tagging in other specification documents. The ECMAScript Language Specification has optional support for ECMA-402 (internationalization). The impact of ECMA-402 is called out explicitly in several places. POSIX tags functions as supported in base, XSI, or in one of many option groups.

V. Design decisions

Even more so than for a hosted implementation, systems and embedded programmers do not want to pay for what they don't use. As a consequence, I am avoiding all features that require global storage, even if that storage is immutable.

- Exceptions either require external jump tables or extra bookkeeping instructions.
- RTTI requires extra data in vtables and extra classes that are difficult to optimize away.
- The heap is a big set of global state. In addition, heap exhaustion is typically expressed via exception. Some embedded systems don't even have a heap.
- Thread-local storage requires extra code in the operating system for support. In addition, if one thread uses thread local storage, that cost is imposed on other threads.
- Many embedded systems don't have floating point hardware, and software emulated floating point can drag in large runtimes that are difficult to optimize away.
- Some functions require global or thread-local storage. These functions have been omitted or removed from the freestanding library. Examples are the locale aware functions, many of the "exit" functions, and the C random number functions.

There are a few classes that mostly work in a systems programming environment, but have a few member functions that are troublesome. In these situations, the member function may be omitted while keeping the rest.

- `array::at()`'s whole reason of existing is to provide a throwing operator`[]`. `array` will be supported, but without the `at()` member function.
- `error_code` and `error_condition` will be supported in general, but the non-member operator`<<`, and the member `message()` functions will not, as they require the usage of classes that won't be supported in a freestanding environment.

For the `<algorithms>` header, we would only be able to support sequential execution of parallel algorithms. Since this adds little value, the execution policy overloads will be omitted.

This proposal doesn't remove problematic features from the language, but it does make it so that the freestanding standard library doesn't require those features. Users that disable the problematic features (as is existing practice) will still have portable portions of the standard library at their disposal.

VI. Technical Specifications

Facilities no longer required for freestanding implementations

Usages of floats

- `<cfloat>` entire header
- `<limits>` `numeric_limits<float>`
- `<limits>` `numeric_limits<double>`
- `<limits>` `numeric_limits<long double>`

Usages of global state

- `<cstdlib>` `atexit`
- `<cstdlib>` `at_quick_exit`
- `<cstdlib>` `exit`
- `<cstdlib>` `quick_exit`

Usages of exceptions and RTTI

- `<new>` `bad_alloc`
- `<new>` `bad_array_new_length`
- `<typeinfo>` entire header
- `<exception>` entire header

The heap

- `<new>` `nothrow_t`
- `<new>` `nothrow`
- `<new>` `[new.delete.single]`
- `<new>` `[new.delete.array]`
- `<new>` `new_handler`
- `<new>` `get_new_handler`
- `<new>` `set_new_handler`

Usages of the operating system

- `<new>` `hardware_destructive_interference_size`
- `<new>` `hardware_constructive_interference_size`

Facilities newly required for freestanding implementations

Portions of <cstdlib>

- div_t
- ldiv_t
- lldiv_t
- EXIT_FAILURE
- EXIT_SUCCESS
- _Exit
- atoi
- atol
- atoll
- bsearch
- qsort
- abs(int)
- abs(long int)
- abs(long long int)
- labs
- div
- ldiv
- lldiv

All the error #defines in <cerrno>, but not errno.

All of <system_error> except for the operator<< stream overloads, error_code::message, error_condition::message, and the system_error class.

All of <utility> and <tuple>.

Portions of <memory>.

- pointer_traits
- align
- 23.10.10, specialized algorithms
- unique_ptr
- default_delete (with implementation defined behavior)
- hash<unique_ptr>

All of <functional> except for 23.14.13, polymorphic function wrappers (i.e. std::function and friends).

All of <ratio>.

All of <execution>.

Portions of <charconv>.

- to_chars_result
- from_chars_result
- to_chars(integral)
- from_chars(integral)

The char_traits class from <string>.

Most of <string_view>. These functions will be **omitted**:

- operator<<
- basic_string_view::at
- basic_string_view::copy
- basic_string_view::substr
- basic_string_view::compare(size_type pos1, size_type n1, basic_string_view s);
- basic_string_view::compare(size_type pos1, size_type n1, basic_string_view s, size_type pos2, size_type n2);
- basic_string_view::compare(size_type pos1, size_type n1, const charT* s);
- basic_string_view::compare(size_type pos1, size_type n1, const charT* s, size_type n2);

Portions of <cstring>.

- memcpy
- memmove
- strcpy
- strncpy
- strcat
- strncat
- memcmp
- strcmp
- strncmp
- memchr
- strchr
- strcspn
- strpbrk
- strrchr
- strspn
- strstr
- memset
- strlen

Portions of <wchar>.

- wcscpy
- wcsncpy
- wmemcpy
- wmemmove
- wcscat
- wcsncat
- wcscmp

- wcsncmp
- wmemcmp
- wcschr
- wcscspn
- wcxpbrk
- wcsrchr
- wcssp
- wcsstr
- wcstok
- wmemchr
- wcslen
- wmemset

All of `<array>` except for `array::at`.

All of `<iterator>` except for the stream iterators and the insert iterators.

Most of `<algorithm>` and `<numeric>`. The `ExecutionPolicy` overloads will not be included. The following functions will be **omitted** due to the usage of temporary buffers:

- `stable_sort`
- `stable_partition`
- `inplace_merge`

Portions of `<random>`. The following portions will be **omitted**:

- `random_device`
- `uniform_real_distribution`
- `exponential_distribution`
- `gamma_distribution`
- `weibull_distribution`
- `extreme_value_distribution`
- `normal_distribution`
- `lognormal_distribution`
- `chi_squared_distribution`
- `cauchy_distribution`
- `fisher_f_distribution`
- `student_t_distribution`
- `piecewise_constant_distribution`
- `piecewise_linear_distribution`

A small portion of `<cmath>` will be present.

- `abs(int)`
- `abs(long int)`
- `abs(long long int)`

A portion of `<stdintypes>` will be present.

- `imaxabs`
- `imaxdiv`
- `abs(intmax_t)`
- `div(intmax_t, intmax_t)`

Facilities that remain required for freestanding implementations

All of `<cstdlib>`, `<climits>`, `<cstdint>`, `<initializer_list>`, `<cstdlibarg>`, `<type_traits>`, `<atomic>`, `<stdalign>`, and `<stdbool>`.

Most of `<limits>`

Portions of `<new>`

- `launder`
- `[new.delete.placement]`

`abort` from `<stdlib>`

Notable omissions

`bitset` is not included because many of its functions throw as part of a range check.

`errno` is not included as it is global state.

Many string functions (`strtol` and family) rely on `errno`.

`assert` is not included as it requires a `stderr` stream.

`<variant>` and `<optional>` could work in a freestanding environment if their interfaces didn't rely on exceptions.

`<cctype>` and `<cwctype>` rely heavily on global locale data.

Changes to shared hosted / freestanding areas

The default allocator should be allowed to invoke implementation defined behavior for freestanding implementations. The intent is for freestanding implementations without a heap to `static_assert`. This will improve diagnostics when a user instantiates a template that is in the STL, but isn't supported in the current freestanding mode.

`default_delete` should be allowed to invoke implementation defined behavior in a freestanding implementation. The intent is to allow `unique_ptr` with custom deleters, while still providing good diagnostics with a `static_assert` in freestanding mode.

VII. Acknowledgements

Thanks to Brandon Streiff, Joshua Cannon, Phil Hindman, and Irwan Djajadi for reviewing this proposal.

Similar work was done in the C++11 timeframe by Lawrence Crowl and Alberto Ganesh Barbati in [N3256](#).

CppCon talks on getting C++ support in various unusual environments:

[Baker2017] CppCon 2017: Billy Baker "Almost Unlimited Modern C++ in Kernel-Mode Applications"

[Quinn2016] CppCon 2016: Rian Quinn "Making C++ and the STL Work in the Linux / Windows Kernels"